Movie Rental Data Warehouse Design

Prepared by: Group A

Tasneem Ashour – 12323555

Berlanti Masalha – 12323064

Sara Othman – 12323745

Supervisor: Dr. Moamin Abughazala

An-Najah National University

Faculty of Information Technology and Artificial Intelligence

Department of Computer Science

May 16, 2026

Table of Contents

1. Introduction 2
2. Business Questions 3
3. Dimensional Model Design 4
   3.1. Selected Business Processes 4
   .3.2 Fact Tables 4
   3.3. Dimension Tables 5
4. Dimensional Model Diagram 7
.5 ETL Design 7
   5.1. Extract 7
   5.2. Transform 8
      5.2.1. dim_film 8
      5.2.2. dim_category 8
      5.2.3. bridge_film_category 8
      5.2.4. dim_customer 9
      5.2.5. dim_store 9
      5.2.6. dim_staff 9
      5.2.7. dim_date 9
      5.2.8. fact_rental_film 10
      5.2.9. fact_payment 10
   5.3. Load 10
      5.3.1. Loading Order 10
      5.3.2. Surrogate Key Generation 11
      5.3.3. Connecting Fact Tables to Dimensions 11
      5.3.4. Handling Null Values Before Load 11
      5.3.5. Load Method 11
.6 Data Quality Considerations 12
   6.1. Null Checks on Dimension Tables 12
   6.2. Null Detection and Removal from the Date Dimension 12
   6.3. Duplicate Detection and Removal from the Date Dimension 13
   6.4. Handling Negative days_late Values 13
   6.5. Sentinel Value for Unreturned Films 13
   6.6. Remaining NULL Fill in days_late 13
   6.7. Null Checks on Fact Tables 14
   6.8. Type Consistency for Surrogate Keys 14
7. Conclusion 14

1. Introduction

The purpose of this assignment is to design a high-level data warehouse for a movie rental business based on an existing OLTP (Online Transaction Processing) database schema. The source system, represented by the Sakila movie rental database, supports daily operational activities such as customer registration, film management, inventory control, rental processing, and payment recording. While this structure is efficient for handling transactions, it is not optimized for analytical reporting and business decision-making.

The proposed data warehouse reorganizes the operational data into a dimensional model using a Hybrid Schema that combines Star and Snowflake approaches. This design enables managers to analyze key business areas such as rental trends, revenue generation, customer behavior, film popularity, and store performance over time. The data warehouse stores historical and integrated data, making it suitable for reporting, dashboards, and strategic analysis.

The main difference between the OLTP system and the data warehouse lies in their purpose and structure. The OLTP system is highly normalized and designed for fast insert, update, and delete operations related to day-to-day business transactions. In contrast, the data warehouse is denormalized and organized around fact and dimension tables to support complex queries, aggregations, and trend analysis. OLTP answers operational questions such as “Was this rental completed?”, while the data warehouse answers analytical questions such as “Which films generated the highest revenue this year?”.

2. Business Questions

The proposed data warehouse is designed to support analytical reporting and help business managers make better strategic decisions. The following business questions represent the main types of analysis that the data warehouse should answer.

Business Question	Importance
Which films are rented most frequently?	This question helps identify the most popular films among customers. Understanding film popularity allows the business to improve inventory planning, marketing strategies, and future film acquisitions.
Which films generate the highest revenue?	Analyzing revenue by film helps the company determine which films contribute most to business profit. This information supports pricing decisions and investment in profitable film categories.
Which stores generate the highest number of rentals?	Comparing rental activity between stores helps evaluate store performance and customer demand in different locations. This information can support operational and expansion decisions.
Which stores generate the highest revenue?	Revenue analysis by store allows managers to identify the most profitable locations and evaluate overall business performance across branches.
Which customers rent the most films per month?	This question helps identify loyal and highly active customers. The business can use this information to create loyalty programs, personalized promotions, and customer retention strategies.
Which customers generate the highest revenue?	Analyzing customer spending behavior helps identify high-value customers who contribute significantly to company revenue. This supports customer relationship management and marketing efforts.
How does rental activity change over time?	Time-based analysis helps management identify seasonal trends, busy periods, and changes in customer demand. This information supports forecasting and operational planning.
Which staff members process the highest number of payments?	This analysis helps evaluate employee productivity and operational efficiency. Management can use the results to support performance assessment, workload distribution, employee training, and staff planning decisions.
Which films are returned late most often?	This question helps identify films with frequent late returns, allowing management to review rental policies, improve customer reminders, or adjust rental periods.

These business questions are important because they transform operational data into meaningful analytical insights that support better decision-making, improve operational efficiency, increase profitability, and enhance customer satisfaction.

3. Dimensional Model Design

3.1. Selected Business Processes

The main business processes identified from the OLTP system are:

1. Film Rental Transactions
2. Payment Transactions
3. Customer Rental

These processes became the main fact tables in the warehouse.

3.2. Fact Tables

Fact_Rental_Film

Item	Description
Business Process	Film rental transactions
Grain	One record for each rental transaction
Measures	rental_duration, days_late
Related Dimensions	dim_Film, dim_Store, dim_Date
Source OLTP Tables	rental, inventory, film, store
Purpose	Used to analyze rental activity, late returns, and rental trends.

Fact_Payment

Item	Description
Business Process	Payment transactions
Grain	One record for each payment transaction
Measures	Payment Amount
Related Dimensions	Dim_Customer, Dim_Film, Dim_Store, Dim_Staff, Dim_date
Source OLTP Tables	payment, rental, inventory, film, customer, store, staff
Purpose	Analyze revenue by customer, film, store, staff, and time

Fact_Rental_Customer

Item	Description
Business Process	Customer rental activity
Grain	One record per customer per month
Measures	total_rental
Related Dimensions	Customer, Date
Source OLTP Tables	Rental, Customer
Purpose	Used to analyze customer rental behavior over time.

3.3. Dimension Tables

Dim_Film

Item	Description
PK	Film_Key
Business Key	Film_id
Main Attributes	title, rating, description, film_length, language
Source OLTP Tables	film

Dim_Category

Item	Description
PK	Category_Key
Business Key	Category_id
Main Attributes	Category Name
Source OLTP Tables	category

Dim_Customer

Item	Description
PK	Customer_Key
Business Key	Customer _id
Main Attributes	Full Name, Email, ccity, ccountry, cpostel_code, caddress, cdistrict
Source OLTP Tables	customer, address, city, country

Dim_Date

Item	Description
PK	Date_Key
Main Attributes	full_date, day, month, quarter, year
Source OLTP Tables	Derived from rental, and payment dates

Dim_Store

Item	Description
PK	Store_Key
Business Key	Store _id
Main Attributes	manager_name, saddress, scity, scountry, sdistrict, spostel_code
Source OLTP Tables	store, address, city ,country, staff

Dim_Staff

Item	Description
PK	Staff_Key
Business Key	Staff _id
Main Attributes	full_name, email, staff_store_key
Source OLTP Tables	Staff, Store

Bridge Table: Bridge_Film_Category

Attribute	Description	Key Type
Bridge_key	Unique identifier	Primary key
film_key	Related film key	Foreign Key
category_key	Related category key	Foreign Key

Shared Dimensions

The following dimensions are shared between multiple fact tables:

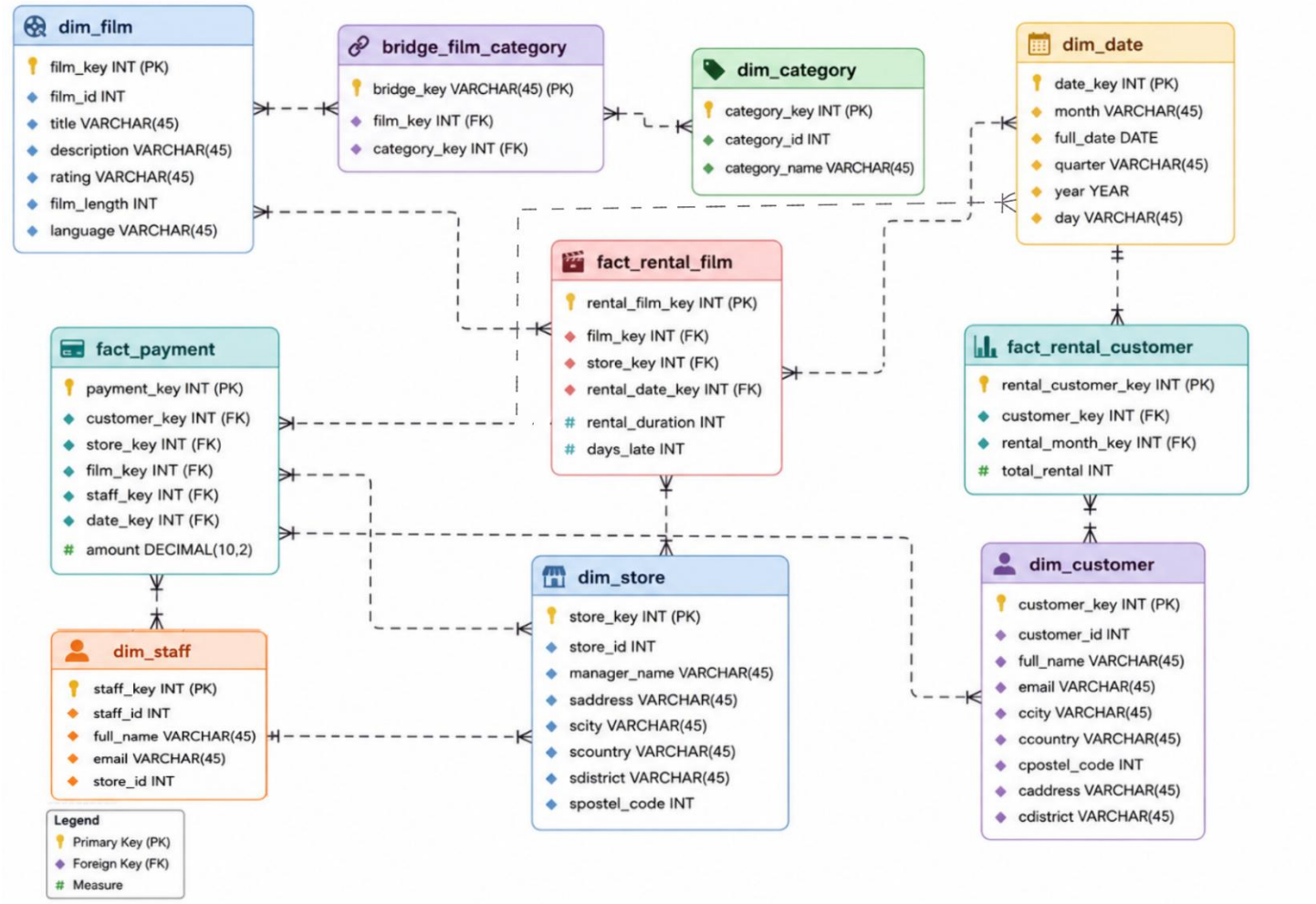
Dimension	Shared Between
dim_customer	fact_payment, fact_rental_customer
dim_film	fact_payment, fact_rental_film
dim_store	fact_payment, fact_rental_film
dim_date	fact_payment, fact_rental_film, fact_rental_customer

Schema Type

The proposed design uses a Hybrid Schema:

- Most dimensions follow a Star Schema design.
- The relationship between films and categories uses a bridge table (bridge_film_category) because one film may belong to multiple categories.

4. Dimensional Model Diagram



5. ETL Design

The ETL (Extract, Transform, Load) process is the backbone of any data warehouse implementation. It is responsible for moving data from the operational OLTP system into the analytical data warehouse in a clean, consistent, and structured format. In this project, the ETL pipeline was implemented using Python with the Pandas library and SQLAlchemy to connect to both the source (OLTP) and destination (Data Warehouse) MySQL databases.

5.1. Extract

The first phase of the ETL process involves reading raw data from the OLTP source database (**filedb**) into memory as Pandas **DataFrames**. The following OLTP tables were extracted:

film	**Core attributes of each film (title, description, rating, length)**
Language	Language name linked to each film
category	Film genre/category classification
film_category	Junction table resolving the many-to-many film–category relationship

customer	Customer personal and account information
address	Physical address linked to customers, stores, and staff
City	City names linked to addresses
country	Country names linked to cities
Store	Store-level information including manager reference
staff	Staff member details and store assignment
Inventory	Links each physical copy of a film to a specific store
rental	Records of individual rental transactions (rental date, return date)
Payment	Records of payment transactions tied to rentals

All tables were extracted using the following pattern:

```
table = pd.read_sql("SELECT * FROM table_name", con=engine1)
```

Both the source OLTP database and the destination data warehouse database were connected to using SQLAlchemy engine objects:

```
engine1 = create_engine(srcConnection_string)   # OLTP source
engine2 = create_engine(dstConnection_string)   # Data Warehouse destination
```

5.2. Transform

The transformation phase is where raw operational data is cleaned, restructured, enriched, and reorganized to fit the dimensional model of the data warehouse. The following transformations were applied for each dimension and fact table:

5.2.1. dim_film

- The **film** table was joined with the **language** table using **language_id** to resolve the language name.
- Columns were renamed for clarity (**name** → **language**).
- Irrelevant operational columns (such as **special_features, last_update**) were excluded.
- The final dimension retains: **film_id, title, description, rating, film_length**, and **language.**

5.2.2. dim_category

- Extracted directly from the **category** OLTP table with minimal transformation.
- The column name was renamed to **category_name** for clarity.

5.2.3. bridge_film_category

- The **film_category** junction table was used to resolve the many-to-many relationship between films and categories.
- Business keys (**film_id, category_id**) were replaced with warehouse surrogate keys (**film_key, category_key**) by joining back to the already-loaded **dim_film** and **dim_category** tables in the data warehouse.

5.2.4. dim_customer

- The **customer** table was joined with address, city, and country to construct a complete, denormalized customer record.
- First name and last name were concatenated into a single **full_name** attribute.
- Location attributes **(city, country, address, district, postal code)** were flattened into the dimension row.
- Unnecessary operational fields **(e.g., active, create_date, last_update)** were dropped.

5.2.5. dim_store

- The **store** table was joined with staff (to get the manager's name), **address**, **city**, and **country**.
- The manager's first and last names were concatenated into **manager_name**.
- Location attributes were denormalized directly into the store record.

5.2.6. dim_staff

- The **staff** table was joined with the **dim_store** warehouse table to attach the store_key surrogate key.
- First and last names were concatenated into **full_name**.

5.2.7. dim_date

The date dimension was constructed by collecting all relevant date values from the operational data:

- **rental_date** from the **rental** table
- **return_date** from the **rental** table
- **payment_date** from the **payment** table

The following transformations were applied:

- Normalization: The **.dt.normalize()** function was applied to strip the time component from all datetime values, retaining only the date portion (e.g., **2005-05-24 22:53:30** → **2005-05-24**). This ensures the date dimension contains one row per unique calendar date.
- Null handling: NULL values (primarily from **return_date** for unreturned films) were dropped using **dropna()**.
- Deduplication: Duplicate dates were removed using **drop_duplicates()**.
- Attribute derivation: From each unique date, the following attributes were derived:
  - **month** (full month name)
  - **quarter** (e.g., Q2)
  - **year** (four-digit),
  - **day** (full day name).

5.2.8. fact_rental_film

- The **rental** table was joined with inventory to obtain **film_id** and **store_id** for each rental transaction.
- The **film** table was joined to retrieve the **rental_duration** (expected return period in days).

Actual rental duration was calculated as:

```
actual_duration = return_date − rental_date  (in days)
```

Late return days were calculated as:

```
days_late = actual_duration − rental_duration
```

- If the result was negative (returned early), days_late was set to 0.
- If return_date was NULL (film not yet returned), days_late was set to −1 as a sentinel value

- Business keys (**film_id**, **store_id**, **rental date**) were replaced with surrogate keys (**film_key**, **store_key**, **date_key**) by joining with the loaded dimension tables in the warehouse.

5.2.9. fact_payment

- The **payment** table was joined with **rental** to get **inventory_id**, and then with inventory to derive **film_id** and **store_id**.
- Surrogate keys were attached by joining with **dim_customer**, **dim_store**, **dim_film**, **dim_staff**, and **dim_date**.
- The payment date was normalized before joining with **dim_date** to match on the date portion only.
- The key measure retained is **amount** (payment amount in dollars).

5.3. Load

The load phase inserts the transformed data into the destination data warehouse database (**movie_rental**). Loading follows a strict order to satisfy foreign key dependencies and ensure referential integrity.

5.3.1. Loading Order

Dimension tables must be loaded before fact tables, because fact tables reference the surrogate keys generated in the dimensions. The loading order used in this project was:

1. dim_film
2. dim_category
3. bridge_film_category
4. dim_customer
5. dim_store
6. dim_staff
7. dim_date
8. fact_rental_film
9. fact_payment

5.3.2. Surrogate Key Generation

Surrogate keys were generated automatically using the Pandas DataFrame index. Each dimension table was assigned an index named after its surrogate key (e.g., **film_key**, **customer_key**, **date_key**), and this index was written to the database alongside the data using **index=True** in **the to_sql()** call:

```
dim_film.index.name = "film_key"
dim_film.to_sql("dim_film", con=engine2, if_exists="replace", index=True)
```

This approach ensures that each row in a dimension receives a unique, sequential integer key that has no business meaning — the core principle of surrogate key design.

5.3.3. Connecting Fact Tables to Dimensions

Once dimensions were loaded, their surrogate keys were read back from the warehouse and used to replace business keys in the fact tables. For example:

```
dim_film_dw = pd.read_sql("SELECT film_id, film_key FROM dim_film", con=engine2)
fact_rental_film = rf.merge(dim_film_dw, on="film_id", how="left")
```

This pattern ensures that fact tables store only surrogate keys (integers), not descriptive business keys. Only the columns needed in the final fact table (foreign keys and measures) were retained before loading.

5.3.4. Handling Null Values Before Load

- NULL **actual_duration** values (for unreturned films) were intentionally preserved in **fact_rental_film**, since computing a duration without a return date would be incorrect.
- NULL **days_late** values were filled with 0 using **fillna(0)** where applicable.

All surrogate key columns were cast to integer type before loading to avoid type inconsistencies:

```
fact_rental_film = fact_rental_film.astype({
    "film_key": int,
    "store_key": int,
    "date_key": int
})
```

5.3.5. Load Method

All tables were loaded using the Pandas **to_sql()** method with **if_exists="replace"**, which drops and recreates the target table on each run. This approach is appropriate for a full-refresh ETL pipeline where the entire warehouse is rebuilt from scratch in each execution cycle:

```
fact_rental_film.to_sql(
    "fact_rental_film",
    con=engine2,
    if_exists="replace",
    index=True
```

This strategy guarantees that the data warehouse always reflects the latest state of the OLTP source without accumulating stale or duplicate records.

6. Data Quality Considerations

Data quality is an essential part of the ETL process because incorrect or incomplete data can lead to inaccurate analysis and unreliable business decisions. During the ETL implementation, several data quality rules were applied to ensure that the data loaded into the warehouse is clean, consistent, and valid.

6.1. Null Checks on Dimension Tables

After constructing each dimension table from the OLTP source data, the ETL process checks for NULL values in all columns using isnull().sum(). This validation was applied to tables such as dim_film, dim_category, dim_store, and dim_staff. The purpose of this step is to detect incomplete records before loading them into the warehouse.

```
print(dim_film.isnull().sum())
print(dim_category.isnull().sum())
print(dim_store.isnull().sum())
print(dim_staff.isnull().sum()))
```

6.2. Null Detection and Removal from the Date Dimension

The dim_date table was created by collecting dates from rental_date, return_date, and payment_date. Since some films were not yet returned, NULL values appeared in return_date. The ETL process first checks the number of NULL values and then removes them using dropna() to ensure that only valid dates exist in the date dimension.

```
print("null")
print(all_dates.isnull().sum())

all_dates.dropna(inplace=True)
```

6.3. Duplicate Detection and Removal from the Date Dimension

Because date values were collected from multiple columns and tables, duplicate dates appeared frequently. The ETL process detects duplicates using duplicated().sum() and removes them using drop_duplicates() so that each date appears only once in dim_date.

```
print("duplicated")
print(all_dates.duplicated().sum())

all_dates.drop_duplicates(inplace=True)

print(all_dates.duplicated().sum())
print(all_dates.isnull().sum())
```

6.4. Handling Negative days_late Values

When customers return films before the expected due date, the calculated days_late value becomes negative. Since negative late days have no business meaning, the ETL process replaces these values with 0.

```
for i in rf.index:
    if rf.loc[i, 'days_late'] < 0:
        rf.loc[i, 'days_late'] = 0
```

6.5. Sentinel Value for Unreturned Films

Films that have not yet been returned contain NULL values in return_date, which makes it impossible to calculate days_late. Instead of leaving these values as NULL, the ETL process assigns a sentinel value of -1 to clearly indicate active rentals.

```
for i in rf.index:
    if pd.isnull(rf.loc[i, 'return_date']):
        rf.loc[i, 'days_late'] = -1
```

6.6. Remaining NULL Fill in days_late

After applying the previous cleaning rules, any remaining NULL values in days_late are replaced with 0 using fillna(0). This ensures that no NULL values remain before loading the fact table into the warehouse.

```
rf['days_late'] = rf['days_late'].fillna(0)
```

6.7. Null Checks on Fact Tables

After constructing the fact tables and attaching surrogate keys from the dimensions, the ETL process checks again for NULL values using isnull().sum(). This validation ensures that all joins were successful and that every fact record is connected correctly to dimension tables.

```
print(fact_rental_film.isnull().sum())
print(fact_payment.isnull().sum())
```

6.8. Type Consistency for Surrogate Keys

Before loading the fact tables into the data warehouse, surrogate key columns are converted to integer type using astype(int). This prevents datatype mismatches between foreign keys in fact tables and primary keys in dimension tables.

```
fact_rental_film = fact_rental_film.astype({
    'film_key': int,
    'store_key': int,
    'date_key': int
})
```

7. Conclusion

This project successfully transformed the Movie Rental OLTP database into an analytical data warehouse using dimensional modeling and a Python-based ETL pipeline. The warehouse was designed mainly using a Star Schema structure with a hybrid approach for handling the many-to-many relationship between films and categories through a bridge table.

Several dimension tables and fact tables were created to support analytical reporting, including rental activity, revenue analysis, customer behavior, store performance, and late return analysis. The ETL process extracted data from the OLTP system, transformed and cleaned the data, generated surrogate keys, and loaded the final data into the warehouse using Pandas and SQLAlchemy.

To improve data reliability, multiple data quality rules were applied during ETL, including NULL checks, duplicate removal, type consistency validation, and handling invalid values such as negative late days. The warehouse also supported the creation of analytical visualizations such as top rented films, revenue by store, rental trends over time, and customer activity analysis.

The proposed solution helps business stakeholders answer important analytical questions more efficiently than the original OLTP system. It supports business decision-making by providing clean, organized, and analysis-ready data for evaluating customer behavior, film popularity, revenue generation, staff performance, and rental trends.

Overall, the project demonstrates practical implementation of data warehouse design, ETL development, data quality management, and business analytics using a realistic movie rental scenario.